

```

import * as RomiCore from '@osrf/romi-js-core-interfaces';
import React from 'react';

export interface DoorProps {
  door: RomiCore.Door;
  doorState?: RomiCore.DoorState;
  onOpenClick?(e: React.MouseEvent): void;
  onCloseClick?(e: React.MouseEvent): void;
}

export const Door = (props: DoorProps) => {
  const { door, doorState, onOpenClick, onCloseClick } = props;
  return (
    <div>
      Door: {door.name}
      <br />
      State: {doorState ? doorState.current_mode.value : 'Unknown'}
      <br />
      <button onClick={e => onOpenClick && onOpenClick(e)}>Open</button>
      <button onClick={e => onCloseClick && onCloseClick(e)}>Close</button>
      <br />
      <br />
    </div>
  );
};

export default Door;

```

Now we can test it by passing the doors as props. Your `App.tsx` component should now look like this:

```

import * as RomiCore from '@osrf/romi-js-core-interfaces';
import { SossTransport } from '@osrf/romi-js-soss-transport';
import * as jwt from 'jsonwebtoken';
import React from 'react';
import Door from './Door';

function App() {
  const [doors, setDoors] = React.useState<RomiCore.Door[]>([]);

  React.useEffect(() => {
    (async () => {
      const token = jwt.sign({ user: 'example-user' }, 'rmf', { algorithm: 'HS256'
    });
      const transport = await SossTransport.connect('example',
        'wss://localhost:50001', token);
      const buildingMap = (await transport.call(RomiCore.getBuildingMap,
        {})).building_map;
      setDoors(buildingMap.levels.flatMap((level) => level.doors));
    })();
  }, []);

  return (
    <React.Fragment>
      {doors.map((door) => (
        <Door door={door} />
      ))}
    </React.Fragment>
  );
}

export default App;

```

Don't worry about the door state for now. If everything goes well, you should see 3 doors that are in the building listed:

Door: main_door
State: Unknown

Open Close

Door: coe_door
State: Unknown

Open Close

Door: hardware_door
State: Unknown

Open Close

Listening for Door States

Previously we managed to render the list of doors in the RMF system, but the building map doesn't tell us the doors' states, so let's fix that. First, let's add a React state to track the door states:

```
const [doorStates, setDoorStates] = React.useState<Record<string,
RomiCore.DoorState>>({});
```

The door states can be obtained by subscribing to the `door_states` topic. Add the following to your effect:

```
transport.subscribe(RomiCore.doorStates, (doorState) =>
  setDoorStates((prev) => ({ ...prev, [doorState.door_name]: doorState })),
);
```

This performs a ROS 2 subscription to the `RomiCore.doorStates` topic. Similar to the service call we did earlier, `romi-js` abstracts away the ROS 2 topic name and provides type information. The callback will be fired each time a new door state message comes in. In the callback, we simply update the `doorStates` state.

Now just pass the door state to the door component:

```
<Door door={door} doorState={doorStates[door.name]} />
```

The end result of your `App.tsx` should look like this:

```

import * as RomiCore from '@osrf/romi-js-core-interfaces';
import { SossTransport } from '@osrf/romi-js-soss-transport';
import * as jwt from 'jsonwebtoken';
import React from 'react';
import Door from './Door';

function App() {
  const [doors, setDoors] = React.useState<RomiCore.Door[]>([]);
  const [doorStates, setDoorStates] = React.useState<Record<string,
RomiCore.DoorState>>({});

  React.useEffect(() => {
    (async () => {
      const token = jwt.sign({ user: 'example-user' }, 'rmf', { algorithm: 'HS256'
});
      const transport = await SossTransport.connect('example',
'wss://localhost:50001', token);
      const buildingMap = (await transport.call(RomiCore.getBuildingMap,
{})).building_map;
      setDoors(buildingMap.levels.flatMap((level) => level.doors));

      transport.subscribe(RomiCore.doorStates, (doorState) =>
        setDoorStates((prev) => ({ ...prev, [doorState.door_name]: doorState })),
      );
    })();
  }, []);

  return (
    <React.Fragment>
      {doors.map((door) => (
        <Door door={door} doorState={doorStates[door.name]} />
      ))}
    </React.Fragment>
  );
}

export default App;

```

And just like that we now have the door states!

Door: main_door

State: 0

Open

Close

Door: coe_door

State: 0

Open

Close

Door: hardware_door

State: 0

Open

Close

The door states are numbers like 0, 1 and 2. This is because RMF uses a constant to represent door states. We could run a simple function to convert these constants into strings:

```
function doorModeString(doorMode: RomiCore.DoorMode): string {  
  switch (doorMode.value) {  
    case 2:  
      return 'Open';  
    case 0:  
      return 'Closed';  
    case 1:  
      return 'Moving';  
    default:  
      return 'Unknown';  
  }  
}
```

But how do we know 2 means "Open" etc? We can find out by reading the RMF manual or inspecting the ROS 2 message definitions, but we can do better with `RomiCore`. It provides the list of constants in a more readable form:

```
function doorModeString(doorMode: RomiCore.DoorMode): string {  
  switch (doorMode.value) {  
    case RomiCore.DoorMode.MODE_OPEN:  
      return 'Open';  
    case RomiCore.DoorMode.MODE_CLOSED:  
      return 'Closed';  
    case RomiCore.DoorMode.MODE_MOVING:  
      return 'Moving';  
    default:  
      return 'Unknown';  
  }  
}
```

With this it's obvious what each constant represents so we don't have to refer to anything else to find its meaning.

Go ahead and add this to your `Door.tsx`. It should look like this now:

```

import * as RomiCore from '@osrf/romi-js-core-interfaces';
import React from 'react';

export interface DoorProps {
  door: RomiCore.Door;
  doorState?: RomiCore.DoorState;
  onOpenClick?(e: React.MouseEvent): void;
  onCloseClick?(e: React.MouseEvent): void;
}

export const Door = (props: DoorProps) => {
  const { door, doorState, onOpenClick, onCloseClick } = props;
  const modeString = doorState ? doorModeString(doorState.current_mode) :
'Unknown';
  return (
    <div>
      Door: {door.name}
      <br />
      State: {modeString}
      <br />
      <button onClick={e => onOpenClick && onOpenClick(e)}>Open</button>
      <button onClick={e => onCloseClick && onCloseClick(e)}>Close</button>
      <br />
      <br />
    </div>
  );
};

function doorModeString(doorMode: RomiCore.DoorMode): string {
  switch (doorMode.value) {
    case RomiCore.DoorMode.MODE_OPEN:
      return 'Open';
    case RomiCore.DoorMode.MODE_CLOSED:
      return 'Closed';
    case RomiCore.DoorMode.MODE_MOVING:
      return 'Moving';
    default:
      return 'Unknown';
  }
}

export default Door;

```

Great! Now we have readable door states instead of cryptic numbers.

Door: main_door

State: Closed

Open

Close

Door: coe_door

State: Closed

Open

Close

Door: hardware_door

State: Closed

Open

Close

Sending Door Requests

As you may have expected by now, all we have to do here is to send door requests to RMF.

First, create a publisher. Add this to the start of the render function:

```
const doorRequestPub = React.useRef<RomiCore.Publisher<RomiCore.DoorRequest> | null>(null);
```

Then add this helper function:

```
const requestDoor = (door: RomiCore.Door, mode: number) => {  
  if (doorRequestPub.current) {  
    const request: RomiCore.DoorRequest = {  
      door_name: door.name,  
      request_time: RomiCore.toRosTime(new Date()),  
      requested_mode: { value: mode },  
      requester_id: 'example-request',  
    };  
    doorRequestPub.current.publish(request);  
  }  
};
```

It takes in a `RomiCore.Door` and a number representing the desired mode, and crafts a `RomiCore.DoorRequest` message and sends it using the publisher. Normally you would have to consult the RMF manual or the ROS 2 definitions to know exactly what you need to send. Again, `RomiCore` provides the typing information to make it easier to fill in the required fields.

Finally, add this to the props passed to the door component:

```
onOpenClick={() => requestDoor(door, RomiCore.DoorMode.MODE_OPEN)}  
onCloseClick={() => requestDoor(door, RomiCore.DoorMode.MODE_CLOSED)}
```

Your final `App.tsx` should look like this:

```

import * as RomiCore from '@osrf/romi-js-core-interfaces';
import { SossTransport } from '@osrf/romi-js-soss-transport';
import * as jwt from 'jsonwebtoken';
import React from 'react';
import Door from './Door';

function App() {
  const [doors, setDoors] = React.useState<RomiCore.Door[]>([]);
  const [doorStates, setDoorStates] = React.useState<Record<string,
RomiCore.DoorState>>({});
  const doorRequestPub = React.useRef<RomiCore.Publisher<RomiCore.DoorRequest> |
null>(null);

  React.useEffect(() => {
    (async () => {
      const token = jwt.sign({ user: 'example-user' }, 'rmf', { algorithm: 'HS256'
});
      const transport = await SossTransport.connect('example',
'wss://localhost:50001', token);
      const buildingMap = (await transport.call(RomiCore.getBuildingMap,
{})).building_map;
      setDoors(buildingMap.levels.flatMap((level) => level.doors));

      transport.subscribe(RomiCore.doorStates, (doorState) =>
        setDoorStates((prev) => ({ ...prev, [doorState.door_name]: doorState })),
      );

      doorRequestPub.current =
transport.createPublisher(RomiCore.adapterDoorRequests);
    })();
  }, []);

  const requestDoor = (door: RomiCore.Door, mode: number) => {
    if (doorRequestPub.current) {
      const request: RomiCore.DoorRequest = {
        door_name: door.name,
        request_time: RomiCore.toRosTime(new Date()),
        requested_mode: { value: mode },
        requester_id: 'example-request',
      };
      doorRequestPub.current.publish(request);
    }
  };

  return (
    <React.Fragment>
      {doors.map((door) => (
        <Door
          door={door}
          doorState={doorStates[door.name]}
          onOpenClick={() => requestDoor(door, RomiCore.DoorMode.MODE_OPEN)}
          onCloseClick={() => requestDoor(door, RomiCore.DoorMode.MODE_CLOSED)}
        />

```